Verzweigungen - if

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Operatoren
- Ausdrücke / Anweisungen
- Typanpassung

Was wird in diesem Foliensatz besprochen?

- Verzweigungen in Java
- Varianten von if-else

Was lernen Sie kennen?

- Anweisungen in Java
- if-else-Anweisung
- Mehrfachverzweigungen
- Programmiermuster mit if/else

Elementare Anweisungen

Zuweisung

```
Variable = Wert;
```

Ausdrucksanweisung

```
Ausdruck;
```

- Nur sinnvoll, wenn der Ausdruck Seiteneffekte hat (z. B. i++)

Block

- Zusammenfassung von aufeinander folgenden Anweisungen

```
{
  Anweisung1;
  Anweisung2;
  …
}
```

Leere Anweisung

```
;
```

Motivation für Verzweigungen

- Bisher wurden die einzelnen Zeilen eines Programms sequentiell abgearbeitet
- Sehr oft sollen aber bestimmte Anweisungen situationsabhängig ausgewählt werden
- Mit Verzweigungen kann der Ablauf des Programms situationsabhängig gesteuert werden

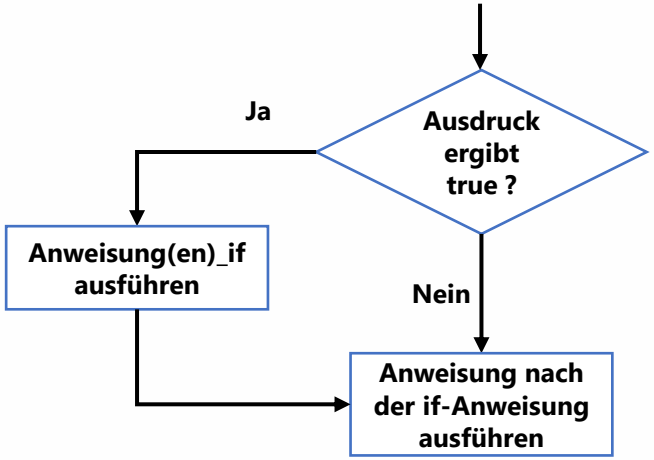
if-Anweisung

Allgemeine Form

```
if (Ausdruck)
  Anweisung(en)_if
```

- Anmerkung
  - Ausdruck in Klammern muss einen booleschen Wert erzeugen

Ablauf



Vergleichsoperatoren (einfache Datentypen)

Java - Notation	Mathematische Notation	Beispiel für true	Beispiel für false
a < b	a < b	2 < 13	2 < 2
a > b	a > b	13 > 2	2 > 13
a <= b	a ≤ b	2 <= 2	3 <= 2
a >= b	a ≥ b	3 >= 2	2 >= 3
a == b	a = b	2 == 2	2 == 3
a != b	a ≠ b	2 != 3	2 != 2

Vergleiche liefern einen booleschen Wert (true, false)

Beispiel (if)

```java
public class IfTest1 {

    public static void main(String[] args) {
        int first = (int) (Math.random() * 100);
        int second = (int) (Math.random() * 100);
        if (first > second) {
            System.out.println("first > second");
        }
        System.out.println("first = " + first);
        System.out.println("second = " + second);
    }

}
```

```
first = 39
second = 75
```

```
first > second
first = 64
second = 49
```

Die Methode random der Klasse Math liefert einen Wert (Zufallszahl) größer oder gleich 0 aber kleiner als 1.0 (vom Typ double) zurück

Hinweis – Klasse Math

Hilfsklasse für mathematische Operationen

- abs, min, max, sin, cos, log, pow, sqrt …
- Enthält auch Konstanten E (Eulersche Zahl) und PI

API

- https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Math.html

Hinweis zu Einrückungen

Erhöhen die Lesbarkeit

- Programmstruktur sichtbar machen

Typische Einrückungstiefe

- 4 Leerzeichen pro Ebene

if-Anweisungen einzeilig schreiben (nur sparsam verwenden)

```
if (n != 0) x = x/n;
```

if – Mehrere Anweisungen

- Bei mehreren Anweisungen Block verwenden
- Beispiel für Unterschied im Ergebnis

```java
int x = 5;
int y = 3;
if (x < 10) {
    System.out.println(x + y);
    System.out.println(x - y);
}
if (x < 10)
    System.out.println(x + y);
    System.out.println(x - y);
```

8
2
8
2

Absichtlich falsche Formatierung

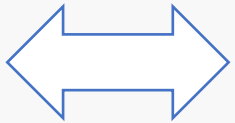
```java
int x = 15;
int y = 3;
if (x < 10) {
    System.out.println(x + y);
    System.out.println(x - y);
}
if (x < 10)
    System.out.println(x + y);
    System.out.println(x - y);
```

12

if – Mehrere if-Anweisungen hintereinander

- Implementieren sich gegenseitig ausschließende Ereignisse
- Beispiele (y ist schon vorher belegt worden)

```
…
int x = 0;
if (y < 10) {
    x = 1;
}
if (y >= 10) {
    x = 2;
}
…
```



```
…
int x = 2;
if (y < 10) {
    x = 1;
}
…
```

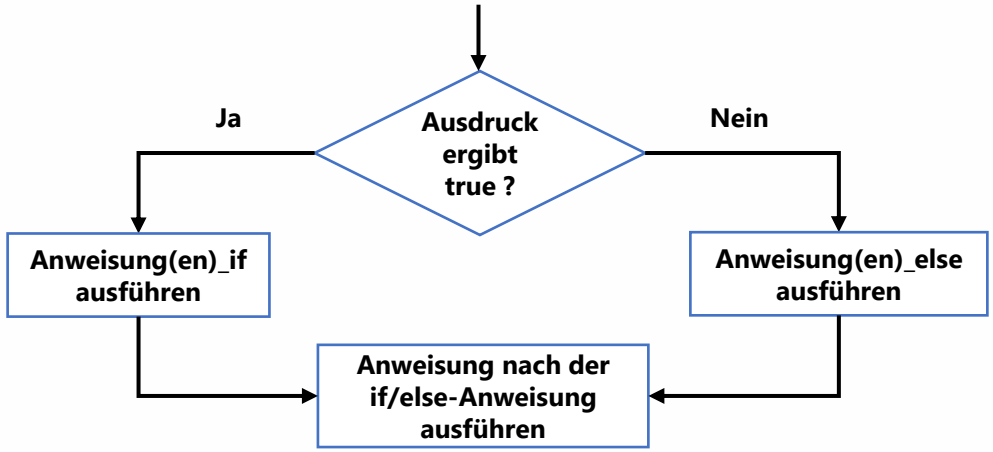
Beide Codefragmente erzeugen dasselbe Ergebnis, d.h. nach der Ausführung gilt
Ist y < 10, dann hat x den Wert 1
Ist y >= 10, dann hat x den Wert 2

if-else

Allgemeine Form

```
if (Ausdruck)
  Anweisung(en)_if
else
  Anweisung(en)_else
```

Ablauf



Beispiel (if-else)

```java
public class IfTest2 {

    public static void main(String[] args) {
        int first = (int) (Math.random() * 100);
        int second = (int) (Math.random() * 100);
        if (first > second) {
            System.out.println("first = " + first);
            System.out.println("second = " + second);
            System.out.println("first > second");
        } else {
            System.out.println("first = " + first);
            System.out.println("second = " + second);
            System.out.println("first <= second");
        }
    }

}
```

```
first = 59
second = 53
first > second
```

```
first = 49
second = 71
first <= second
```

Vergleich if-else und unabhängige if-Anweisungen

```
…
int x = 0;
if (y < 10) {
    x = 1;
} else {
    x = 2;
}
…
```

```
…
int x = 0;
if (y < 10) {
    x = 1;
}
if (y >= 10) {
    x = 2;
}
…
```

```
…
int x = 2;
if (y < 10) {
    x = 1;
}
…
```

Alle drei Codefragmente erzeugen dasselbe Ergebnis, d.h. nach der Ausführung gilt
Ist y < 10, dann hat x den Wert 1
Ist y >= 10, dann hat x den Wert 2

Beispiel (Maximum dreier Zahlen)

2 verschiedene Versionen

- Eine Version mit mehreren verschachtelten if-Anweisungen (MaximumOfThree.java)
- Eine Version mit weniger Vergleichen aber mehr Zuweisungen (MaximumOfThree2.java)

Hinweis

- Längere Programme werden ab jetzt in IntelliJ gezeigt
- Alle Programme werden in TUWEL angeboten

Vergleich der zwei Versionen (1)

```
…
if (firstNumber > secondNumber) {
    if (firstNumber > thirdNumber) {
        maximum = firstNumber;
    } else {
        maximum = thirdNumber;
    }
} else {
    if (secondNumber > thirdNumber) {
        maximum = secondNumber;
    } else {
        maximum = thirdNumber;
    }
}
…
```

Beispiel: Diese Zuweisung wird nur erreicht, falls firstNumber > secondNumber **und** firstnumber > thirdnumber gilt

Benötigt immer 2 Vergleiche und 1 Zuweisung

Vergleich der zwei Versionen (2)

```
…
maximum = firstNumber;
if (secondNumber > maximum) {
    maximum = secondNumber;
}
if (thirdNumber > maximum) {
    maximum = thirdNumber;
}
…
```

Diese if-Anweisungen sind unabhängig voneinander. Es kann eine beliebige Kombination davon ausgeführt werden.

Benötigt immer 2 Vergleiche und im Durchschnitt 2 Zuweisungen

Komplexere Ausdrücke

- Mehrere Vergleiche in einem Ausdruck verknüpfen
- Wird mit Hilfe von logischen Operatoren realisiert
- Beispiel

Textuell: x ist größer als 10 **und** y ist kleiner als 20

Textuell kürzer: x > 10 **und** y < 20

Java Notation: `x > 10 && y < 20`

  - Linker bzw. rechter Vergleich liefern jeweils true oder false

Logische Operatoren

- Beispiele für logische Operatoren
  - Negation: !
  - Oder: || (|)
  - Und: && (&)
  - XOR: ^

a	!a
true	false
false	true

a	b	a && b	a || b	a ^ b
false	false	false	false	false
false	true	false	true	true
true	false	false	true	true
true	true	true	true	false

Beispiele für logische Operatoren

```java
int number = 4;
// .....
if ((number % 2 == 0) && (number % 3 == 0)) {
    System.out.println("Here-1: " + number);
}
if ((number % 2 == 0) || (number % 3 == 0)) {
    System.out.println("Here-2: " + number);
}
```

```
Here-2: 4
```

```java
int number = 6;
// .....
if ((number % 2 == 0) && (number % 3 == 0)) {
    System.out.println("Here-1: " + number);
}
if ((number % 2 == 0) || (number % 3 == 0)) {
    System.out.println("Here-2: " + number);
}
```

```
Here-1: 6
Here-2: 6
```

Teilweise auswerten („*Short circuit evaluation*“)

x && y ist immer false, falls x == false

- Sobald x auf false auswertet, wird die gesamte Auswertung beendet

x || y ergibt immer true, falls x == true

- Sobald x auf true auswertet, wird die gesamte Auswertung beendet

Teilweise auswerten – Robustheit

y darf Berechnungen enthalten, die ungültig sind im Fall

- (x && y) bei x == false
- (x || y) bei x == true

Beispiel

```
(x != 0 && y/x > 2)
```

keine Division durch 0 möglich

Vollständige Auswertung mit & und |

Operatorrangfolge in Java

- Verallgemeinerung von „Punkt- vor Strichrechnung“
- Vereinfachter Überblick
  - https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

Dangling else (1)

- Ein `else`-Zweig bei zwei `if`-Anweisungen
  - Wohin gehört der `else`-Zweig?
- Beispiel (irreführend formatiert)

```
if (counter < 5)
  if (counter % 2 == 0)
    System.out.println("HERE1");
else
  System.out.println("HERE2");
```

Dangling else (2)

- Beispiel (irreführend formatiert)

```
if (counter < 5)
  if (counter % 2 == 0)
    System.out.println("HERE1");
else
  System.out.println("HERE2");
```

`else` gehört zum textuell letzten freien `if` im selben Block
`counter` mit Wert 7 führt zu keiner Ausgabe
`counter` mit Wert 3 führt zur Ausgabe `"HERE2"`

Bedingungsoperator

- Ternärer Operator und rechtsassoziativ
- Form

```
Bedingung ? Ausdruck1 : Ausdruck2;
```

- Beispiel

```
int max, a, b;
…
max = (a > b) ? a : b;
```

```
int max, a, b;
…
if (a > b) {
    max = a;
} else {
    max = b;
}
```

Nur sparsam einsetzen !

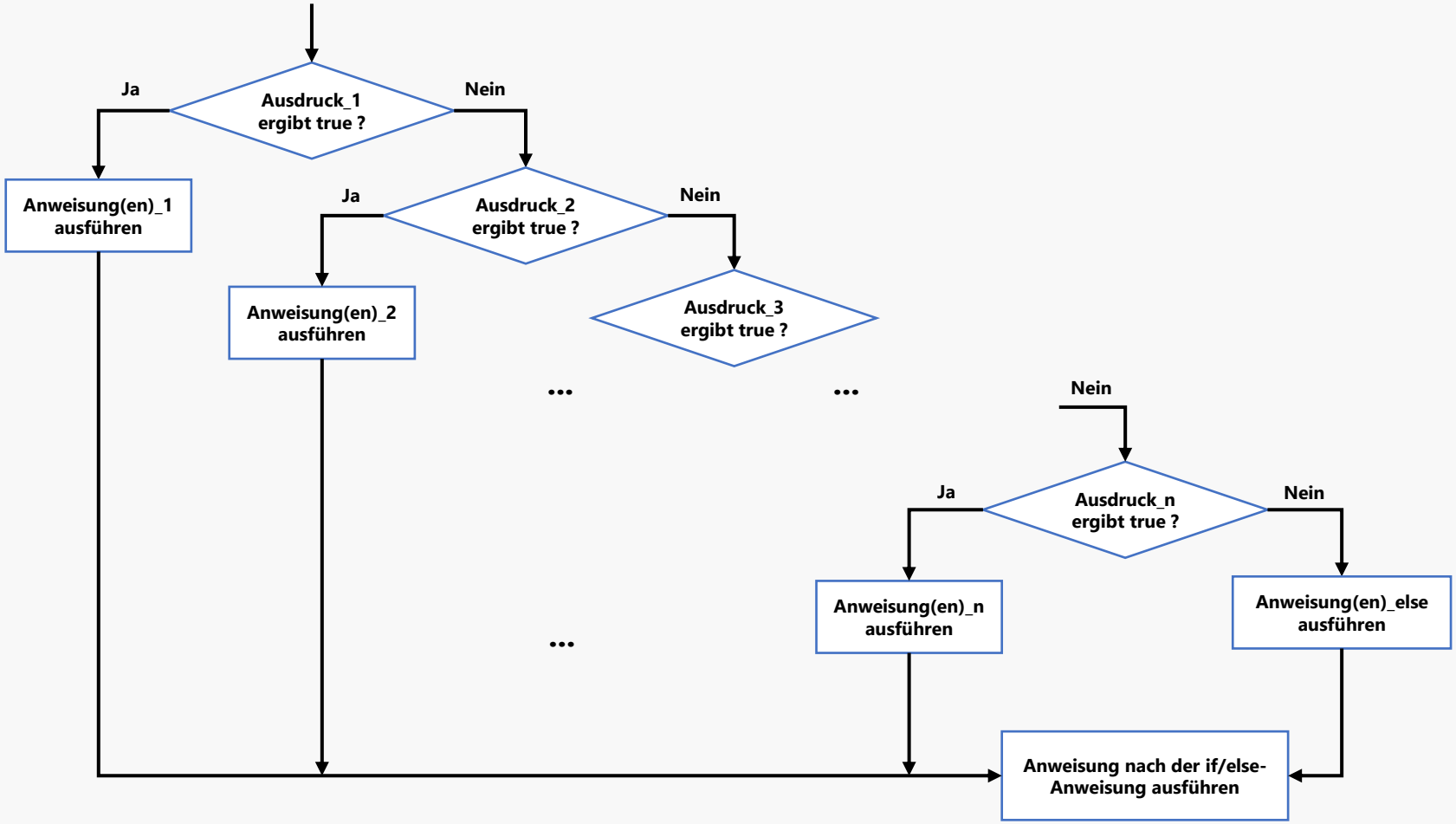
Mehrfachverzweigung bei if

- Allgemeine Form

```
if (Ausdruck_1)
  Anweisung(en)_1
else if (Ausdruck_2)
  Anweisung(en)_2
…
else if (Ausdruck_n)
  Anweisung(en)_n
else
  Anweisung(en)_else
```

- Ein Vergleich wird nach dem anderen durchgeführt (Seiteneffekte beachten!)
- Bei der **ersten Bedingung**, die wahr ist, wird die Anweisung (Anweisungsfolge) abgearbeitet und dann die Anweisung nach der Verzweigung ausgeführt

Mehrfachverzweigung – Ablauf



Beispiel (Mehrfachverzweigung)

```java
public class SequentialChoice {

    public static void main(String[] args) {
        int x = 3;   // Test data
        int result;

        if (x < 0) {
            result = -1;
        } else if (x > 0) {
            result = 1;
        } else {
            result = 0;
        }

        System.out.println(result);
    }

}
```

```
1
```

Beispiel mit verschiedenen Varianten (1)

```java
public class GradingExample1 {

    public static void main(String[] args) {
        int grade = 2;  // Test data

        String text;
        if (grade == 1) {
            text = " excellent ";
        } else if (grade == 2) {
            text = " good ";
        } else if (grade == 3) {
            text = " satisfactory ";
        } else if (grade == 4) {
            text = " sufficient ";
        } else if (grade == 5) {
            text = " insufficient ";
        } else {
            text = "no grade";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: good
```

```java
public class GradingExample1 {

    public static void main(String[] args) {
        int grade = 2;  // Test data

        String text = " no grade ";
        if (grade == 1) {
            text = " excellent ";
        } else if (grade == 2) {
            text = " good ";
        } else if (grade == 3) {
            text = " satisfactory ";
        } else if (grade == 4) {
            text = " sufficient ";
        } else if (grade == 5) {
            text = " insufficient ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

Zweite Version (rechts) ohne else-Zweig - text muss dabei korrekt initialisiert werden

Beispiel mit verschiedenen Varianten (2)

```java
public class GradingExample1 {

    public static void main(String[] args) {
        int grade = 2;  // Test data

        String text = " no grade ";
        if (grade == 1) {
            text = " excellent ";
        }
        if (grade == 2) {
            text = " good ";
        }
        if (grade == 3) {
            text = " satisfactory ";
        }
        if (grade == 4) {
            text = " sufficient ";
        }
        if (grade == 5) {
            text = " insufficient ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: good
```

Unabhängige if-Anweisungen liefern dasselbe Ergebnis

Schlechte Alternative für vorheriges Beispiel

```java
public class GradingExample2 {

    public static void main(String[] args) {
        int grade = 2;  // Test data

        String text;
        text = grade == 1 ? " excellent " :
                grade == 2 ? " good " :
                        grade == 3 ? " satisfactory " :
                                grade == 4 ? " sufficient " :
                                        grade == 5 ? " insufficient " :
                                                " no grade ";

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: good
```

Verschachtelte Bedingungsoperatoren repräsentieren schlechten Stil!

Weiteres Beispiel

- Punkte bestimmen eine Note
  - >= 92 -> excellent
  - >= 83 -> good
  - >= 74 -> satisfactory
  - >= 65 -> sufficient
  - < 65 -> insufficient

Alternatives Beispiel (1)

```java
public class AlternativeGradingExample1 {

    public static void main(String[] args) {
        int points = 82;  // Test data

        String text;
        if (points >= 92) {
            text = " excellent ";
        } else if (points >= 83) {
            text = " good ";
        } else if (points >= 74) {
            text = " satisfactory ";
        } else if (points >= 65) {
            text = " sufficient ";
        } else {
            text = " insufficient ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: satisfactory
```

```java
public class AlternativeGradingExample1 {

    public static void main(String[] args) {
        int points = 82;  // Test data

        String text = "";
        if (points >= 92) {
            text = " excellent ";
        } else if (points >= 83) {
            text = " good ";
        } else if (points >= 74) {
            text = " satisfactory ";
        } else if (points >= 65) {
            text = " sufficient ";
        } else if (points < 65) {
            text = " insufficient ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: satisfactory
```

Alternatives Beispiel (2)

```java
public class AlternativeGradingExample1 {

    public static void main(String[] args) {
        int points = 82;  // Test data

        String text = "";
        if (points >= 92) {
            text = " excellent ";
        }
        if (points >= 83) {
            text = " good ";
        }
        if (points >= 74) {
            text = " satisfactory ";
        }
        if (points >= 65) {
            text = " sufficient ";
        }
        if (points < 65) {
            text = " insufficient ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

Falsch!

```
Your grade: sufficient
```

```java
public class AlternativeGradingExample1 {

    public static void main(String[] args) {
        int points = 82;  // Test data

        String text = "";

        if (points < 65) {
            text = " insufficient ";
        }
        if (points >= 65) {
            text = " sufficient ";
        }
        if (points >= 74) {
            text = " satisfactory ";
        }
        if (points >= 83) {
            text = " good ";
        }
        if (points >= 92) {
            text = " excellent ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: satisfactory
```

if-Anweisungen müssen hier anders angeordnet werden!

Alternatives Beispiel (3)

```java
public class AlternativeGradingExample1 {

    public static void main(String[] args) {
        int points = 82;  // Test data

        String text = "";

        if (points >= 92) {
            text = " excellent ";
        }
        if (points >= 83 && points < 92) {
            text = " good ";
        }
        if (points >= 74 && points < 83) {
            text = " satisfactory ";
        }
        if (points >= 65 && points < 74) {
            text = " sufficient ";
        }
        if (points < 65) {
            text = " insufficient ";
        }


        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: satisfactory
```

Alternatives Beispiel (4)

```java
public class AlternativeGradingExample1 {

    public static void main(String[] args) {
        int points = 82;  // Test data

        String text = "insufficient";

        if (points >= 92) {
            text = " excellent ";
        }
        if (points >= 83 && points < 92) {
            text = " good ";
        }
        if (points >= 74 && points < 83) {
            text = " satisfactory ";
        }
        if (points >= 65 && points < 74) {
            text = " sufficient ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: satisfactory
```

Unabhängige if-Anweisungen können hier beliebig angeordnet werden (mit Ordnung leichter lesbar), aber Bedingungen sind komplizierter!

Klammerstil – 2 bekannte Varianten

OTBS („One True Brace Style“)-Stil (Standard in IntelliJ)

```
if (firstNumber > secondNumber) {
    if (firstNumber > thirdNumber) {
        maximum = firstNumber;
    } else {
        maximum = thirdNumber;
    }
} else {
    …
}
```

+ Mehr Code pro Zeile
+ Schließende Klammer steht unter dem Konstrukt, zu dem sie gehört

Allman-Stil (Standard in Visual Studio)

```
if (firstNumber > secondNumber)
{
    if (firstNumber > thirdNumber)
    {
        maximum = firstNumber;
    }
    else
    {
        maximum = thirdNumber;
    }
}
else
{
    …
}
```

+ Blockbereiche gut erkennbar
- Mehr Zeilen pro Datei

OTBS-Stil

- Vollständige Klammerung
  - Auch bei einzeiligen Blöcken

`if (a < 10)`
`  a = 10;`
	`if (a < 10) {`
	`  a = 10;`
	`}`

  - Rechte Variante braucht mehr Platz aber erleichtert spätere Erweiterung

Reihenfolge der Operanden bei Vergleichen

`if (length == 10) ...`	`if (10 == length) ...`
`if (bytesReceived < bytesExpected) ...`	`if (bytesExpected > bytesReceived) ...`

bevorzugt

Linke Seite	Rechte Seite
Ausdruck, der abgefragt wird und dessen Wert sich ändert	Ausdruck, gegen dessen Wert verglichen wird und der konstant ist (bzw. sich weniger oft ändert)

Programmiermuster mit if/else

Muster	Konstrukt	Beispiel für Form

Beliebige Kombination von Anweisungen mit Überprüfung ausführen	Sequentielle if-Anweisungen	`if (Ausdruck_1)`
		`  Anweisung(en)_1`
		`if (Ausdruck_2)`
		`  Anweisung(en)_2`
		`if (Ausdruck_3)`
		`  Anweisung(en)_3`

Keine oder eine Anweisung mit Überprüfung ausführen	Verschachtelte if-Anweisungen ohne else am Ende	`if (Ausdruck_1)`
		`  Anweisung(en)_1`
		`else if (Ausdruck_2)`
		`  Anweisung(en)_2`
		`else if (Ausdruck_3)`
		`  Anweisung(en)_3`

Genau eine Anweisung mit Überprüfung ausführen	Verschachtelte if-Anweisungen mit else am Ende	`if (Ausdruck_1)`
		`  Anweisung(en)_1`
		`else if (Ausdruck_2)`
		`  Anweisung(en)_2`
		`else`
		`  Anweisung(en)_3`

Reihenfolge der Blöcke

- Beispiel

```
if (a == b) {
    // Erster Fall
} else {
    // Zweiter Fall
}
```

```
if (a != b) {
    // Zweiter Fall
} else {
    // Erster Fall
}
```

- Einfache Empfehlungen
  - Positiver Fall zuerst
  - Einfacher Fall zuerst
  - Interessanter Fall zuerst

Manchmal stehen diese Empfehlungen in Konflikt!